

contains a grounded set of atoms specifying the particular kinship relations that underlie a single story. In other words,  $R$  contains the logical rules that govern all the generated stories in CLUTRR, while  $G$  contains the grounded facts that underlie a specific story.

**Graph generation.** To generate the kinship graph  $G$  underlying a particular story, we first sample a set of gendered<sup>2</sup> entities and kinship relations using a stochastic generation process. This generation process contains a number of tunable parameters—such as the maximum number of children at each node, the probability of an entity being married to another entity, etc.—and is designed to produce a valid, but possibly incomplete “backbone graph”. For instance, this backbone graph generation process will specify “parent”/“child” relations between entities but does not add “grandparent” relations. After this initial generation process, we recursively apply the logical rules in  $R$  to the backbone graph to produce a final graph  $G$  that contains the full set of kinship relations between all the entities.

**Backward chaining.** The resulting graph  $G$  provides the *background knowledge* for a specific story, as each edge in this graph can be treated as a grounded predicate (i.e., fact) between two entities. From this graph  $G$ , we sample the facts that make up the story, as well as the target fact that we seek to predict: First, we (uniformly) sample a target relation  $\mathcal{H}_C$ , which is the fact that we want to predict from the story. Then, from this target relation  $\mathcal{H}_C$ , we run a simple variation of the backward chaining (Gallaire and Minker, 1978) algorithm for  $k$  iterations starting from  $\mathcal{H}_C$ , where at each iteration we uniformly sample a subgoal to resolve and then uniformly sample a KB rule that resolves this subgoal. Crucially, unlike traditional backward chaining, we do not stop the algorithm when a proof is obtained; instead, we run for a fixed number of iterations  $k$  in order to sample a set of  $k$  facts  $\mathcal{B}_C$  that imply the target relation  $\mathcal{H}_C$ .

### 3.3 Adding natural language

So far, we have described the process of generating a conjunctive logical clause  $\mathcal{C} = (\mathcal{H}_C \vdash \mathcal{B}_C)$ , where  $\mathcal{H}_C = [\alpha^*]$  is the target fact (i.e., relation) we seek to predict and  $\mathcal{B}_C = [\alpha_1, \dots, \alpha_k]$  is the set of supporting facts that imply the target relation. We now describe how we convert this logical represen-

<sup>2</sup>Kinship and gender roles are oversimplified in our data (compared to the real world) to maintain tractability.

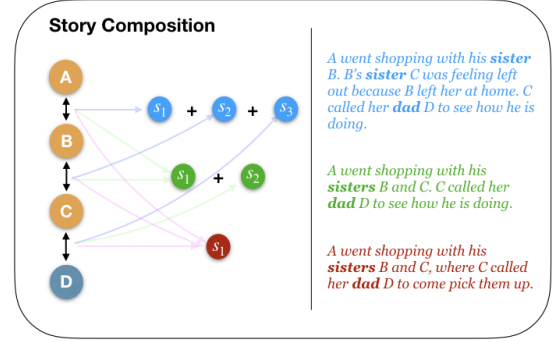


Figure 3: Illustration of how a set of facts can split and combined in various ways across sentences.

tation to natural language through crowd-sourcing. **Paraphrasing using Amazon Mechanical Turk.** The basic idea behind our approach is that we show Amazon Mechanical Turk (AMT) crowd-workers the set of facts  $\mathcal{B}_C$  corresponding to a story and ask the workers to paraphrase these facts into a narrative. Since workers are given a set of facts  $\mathcal{B}_C$  to work from, they are able to combine and split multiple facts across separate sentences and construct diverse narratives (Figure 3). Appendix 1.6 contains further details on our AMT interface (based on the ParLAI framework (Miller et al., 2017)), data collection, and the quality controls we employed. **Reusability and composition.** One challenge for data collection via AMT is that the number of possible stories generated by CLUTRR grows combinatorially as the number of supporting facts increases, i.e., as  $k = |\mathcal{B}_C|$  grows. This combinatorial explosion for large  $k$ —combined with the difficulty of maintaining the quality of the crowd-sourced paraphrasing for long stories—makes it infeasible to obtain a large number of paraphrased examples for  $k > 3$ . To circumvent this issue and increase the flexibility of our benchmark, we reuse and compose AMT paraphrases to generate longer stories. In particular, we collected paraphrases for stories containing  $k = 1, 2, 3$  supporting facts and then replaced the entities from these collected stories with placeholders in order to re-use them to generate longer semi-synthetic stories. An example of a story generated by stitching together two shorter paraphrases is provided below:

[Frank] went to the park with his father, [Brett].  
[Frank] called his brother [Boyd] on the phone.  
He wanted to go out for some beers. [Boyd] went to the baseball game with his son [Jim].  
Q: What is [Brett] and [Jim]’s relationship?

Thus, instead of simply collecting paraphrases for a fixed number of stories, we instead obtain a diverse

Table 1: Statistics of the AMT paraphrases. Jaccard word overlap is calculated within the templates of each individual clause of length  $k$ .

| Number of Paraphrases |          | # clauses |     |
|-----------------------|----------|-----------|-----|
|                       | $k = 1$  | 1,868     | 20  |
|                       | $k = 2$  | 1,890     | 58  |
|                       | $k = 3$  | 2,258     | 236 |
|                       | Total    | 6,016     |     |
| Unique Word Count     |          | 3,797     |     |
| Jaccard Word Overlap  | Unigrams | 0.201     |     |
|                       | Bigrams  | 0.0385    |     |

collection of natural language templates that can be programmatically recombined to generate stories with various properties.

**Dataset statistics.** At the time of submission, we have collected 6,016 unique paraphrases with an average of 19 paraphrases for every possible logical clause of length  $k = 1, 2, 3$ . Table 1 contains summary statistics of the collected paraphrases. Overall, we found high linguistic diversity in the collected paraphrases. For instance, the average Jaccard overlap in unigrams between pairs of paraphrases corresponding to the same logical clause was only 0.201 and only 0.0385 for bigrams. The Appendix contains further examples of the paraphrases.

**Human performance.** To get a sense of the data quality and difficulty involved in CLUTRR, we asked human annotators to solve the task for random examples of length  $k = 2, 3, \dots, 6$ . We found that time-constrained AMT annotators performed well (i.e.,  $> 70\%$ ) accuracy for  $k \leq 3$  but struggled with examples involving longer stories, achieving 40-50% accuracy for  $k > 3$ . However, trained annotators with unlimited time were able to solve 100% of the examples (Appendix 1.7), highlighting the fact that this task requires attention and involved reasoning, even for humans.

### 3.4 Query representation and inference

**Representing the question.** The AMT paraphrasing approach described above allows us to convert the set of supporting facts  $\mathcal{B}_C$  to a natural language story, which can be used to predict the target relation/query  $\mathcal{H}_C$ . However, instead of converting the target query,  $\mathcal{H}_C = [\alpha^*]$ , to a natural language question, we instead opt to represent the target query as a  $K$ -way classification task, where the two entities in the target relation are provided as input and the goal is to classify the relation that holds between

these two entities. This representation avoids the pitfall of revealing information about the answer in the question (Kaushik and Lipton, 2018).

**Representing entities.** When generating stories, entity names are randomly drawn from a set of 300 common gendered English names. Thus, depending on each run, the entities are never the same. This ensures that the entity names are simply placeholders and uncorrelated from the task.

### 3.5 Variants of CLUTRR

The modular nature of CLUTRR provides rich diagnostic capabilities for evaluating the robustness and generalization abilities of neural language understanding systems. We highlight some key diagnostic capabilities available via different variations of CLUTRR below. These diagnostic variations correspond to the concrete datasets that we generated in this work, and we describe the results on these Datasets in Section 4.

**Systematic generalization.** Most prominently, CLUTRR allows us to explicitly evaluate a model’s ability for systematic generalization. In particular, we rely on the following hold-out procedures to test systematic generalization:

- During training, we hold out a subset of the collected paraphrases, and we only use this held-out subset of paraphrases when generating the test set. Thus, to succeed on CLUTRR, an NLU system must exhibit *linguistic generalization* and be robust to linguistic variation at test time.
- We also hold out a subset of the logical clauses during training (for clauses of length  $k > 2$ ).<sup>3</sup> In other words, during training, the model sees all logical rules but does not see all *combinations* of these logical rules. Thus, in addition to linguistic generalization, success on this task also requires *logical generalization*.
- Lastly, as a more extreme form of both logical and linguistic generalization, we consider the setting where the models are trained on stories generated from clauses of length  $\leq k$  and evaluated on stories generated from larger clauses of length  $> k$ . Thus, we explicitly test the ability for models to generalize on examples that require more steps of reasoning than any example they encountered during training.

<sup>3</sup>One should not holdout clauses from length  $k = 2$  in order to allow models to learn the compositionality of all possible binary predicates.

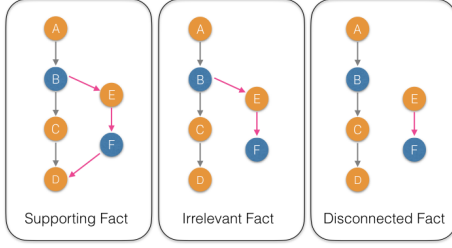


Figure 4: Noise generation procedures of CLUTRR.

**Robust Reasoning.** In addition to evaluating systematic generalization, the modular setup of CLUTRR also allows us to diagnose model robustness by adding *noise facts* to the generated narratives. Due to the controlled semi-synthetic nature of CLUTRR, we are able to provide a precise taxonomy of the kinds of noise facts that can be added (Figure 4). In order to structure this taxonomy, it is important to recall that any set of supporting facts  $\mathcal{B}_C$  generated by CLUTRR can be interpreted as a path,  $p_C$ , in the corresponding kinship graph  $G$  (Figure 2). Based on this interpretation, we view adding noise facts from the perspective of sampling three different types of noise paths,  $p_n$ , from the kinship graph  $G$ :

- *Irrelevant facts:* We add a path  $p_n$ , which has exactly one shared end-point with  $p_C$ . In this way, this is a *distractor* path, which contains facts that are connected to one of the entities in the target relation,  $\mathcal{H}_C$ , but do not provide any information that could be used to help answer the query.
- *Supporting facts:* We add a path  $p_n$ , whose two end-points are on the path  $p_C$ . The facts on this path  $p_n$  are noise because they are not needed to answer the query, but they are supporting facts because they can, in principle, be used to construct alternative (longer) reasoning paths that connect the two target entities.
- *Disconnected facts:* We add paths which neither originate nor end in any entity on  $p_C$ . These disconnected facts involve entities and relations that are completely unrelated to the target query.

## 4 Experiments

We evaluate several neural language understanding systems on the proposed CLUTRR benchmark to surface the relative strengths and shortcomings of these models in the context of inductive reasoning and combinatorial generalization.<sup>4</sup> We aim to answer the following key questions:

- (Q1) How do state-of-the-art NLU models compare in terms of systematic generalization? Can these models generalize to stories with unseen combinations of logical rules?
- (Q2) How does the performance of neural language understanding models compare to a graph neural network that has full access to graph structure underlying the stories?
- (Q3) How robust are these models to the addition of noise facts to a given story?

### 4.1 Baselines

Our primary baselines are neural language understanding models that take unstructured text as input. We consider bidirectional LSTMs (Hochreiter and Schmidhuber, 1997; Cho et al., 2014) (with and without attention), as well as recently proposed models that aim to incorporate inductive biases towards relational reasoning: Relation Networks (RN) (Santoro et al., 2017) and Compositional Memory Attention Network (MAC) (Hudson and Manning, 2018). We also use the large pretrained language model, BERT (Devlin et al., 2018), as well as a modified version of BERT having a trainable LSTM encoder on top of the pretrained BERT embeddings. All of these models (except BERT) were re-implemented in PyTorch 1.0 (Paszke et al., 2017) and adapted to work with the CLUTRR benchmark.

Since the underlying relations in the stories generated by CLUTRR inherently form a graph, we also experiment with a Graph Attention Network (GAT) (Veličković et al., 2018). Rather than taking the textual stories as input, the GAT baseline receives a structured graph representation of the facts that underlie the story.

**Entity and query representations.** We use the various baseline models to encode the natural language story (or graph) into a fixed-dimensional embedding. With the exception of the BERT models, we do not use pre-trained word embeddings and learn the word embeddings from scratch using end-to-end backpropagation. An important note, however, is that we perform Cloze-style anonymization (Hermann et al., 2015) of the entities (i.e., names) in the stories, where each entity name is replaced by a `@entity-k` placeholder, which is randomly sampled from a small, fixed pool of placeholder tokens. The embeddings for these placeholders are randomly initialized and fixed during training.<sup>5</sup>

<sup>4</sup>Code to reproduce all the results in this section will be released at <https://github.com/facebookresearch/clutrr/>.

<sup>5</sup>See Appendix 1.5 for a comparison of placeholder em-